HW5

Zhen Wu

Prob. 6

```
mgs <- function(x) {
  n <- ncol(x)
  m <- nrow(x)
  q <- matrix(0, nrow = m, ncol = n)
  r <- matrix(0, nrow = n, ncol = n)

  for (j in seq_len(n)) {
    v <- x[, j]

    for (i in seq_len(j - 1)) {
      r[i, j] <- t(q[, i]) %*% v
      v <- v - r[i, j] * q[, i]
    }

    r[j, j] <- sqrt(sum(v^2))
    q[, j] <- v / as.numeric(r[j, j])
  }

  q
}

cgs <- function(x) {
  n <- ncol(x)
  m <- nrow(x)
  q <- matrix(0, nrow = m, ncol = n)
  r <- matrix(0, nrow = n, ncol = n)

  for (j in seq_len(n)) {
    v <- x[, j]

    for (i in seq_len(j - 1)) {
      r[i, j] <- t(q[, i]) %*% x[, j]
      v <- v - r[i, j] * q[, i]
    }

    r[j, j] <- sqrt(sum(v^2))
    q[, j] <- v / as.numeric(r[j, j])
  }

  q
}
```

```r
epsilon <- 1e-6
a1 <- c(1, epsilon, 0, 0)
a2 <- c(1, 0, epsilon, 0)
a3 <- c(1, 0, 0, epsilon)
M <- cbind(a1, a2, a3)

qcgs <- cgs(M)
qmgs <- mgs(M)

print(qcgs)
```

```
##       [,1]          [,2]          [,3]
## [1,] 1e+00  7.071696e-07  4.082483e-07
## [2,] 1e-06 -7.071068e-01 -4.082120e-01
## [3,] 0e+00  7.071068e-01 -4.082846e-01
## [4,] 0e+00  0.000000e+00  8.164966e-01
```

```r
print(qmgs)
```

```
##       [,1]          [,2]          [,3]
## [1,] 1e+00  7.071696e-07  4.082846e-07
## [2,] 1e-06 -7.071068e-01 -4.082483e-01
## [3,] 0e+00  7.071068e-01 -4.082483e-01
## [4,] 0e+00  0.000000e+00  8.164966e-01
```

It can easily verified that the output of MGS has better orthogonality than that of CGS.

Prob. 7

To prove the equivalence between the Modified Gram-Schmidt (MGS) method and the Householder QR factorization of the augmented matrix, we proceed by verifying the construction of the Householder reflections P_k and the final transformation matrix P.

First, let $v_k = \begin{bmatrix} -e_k \\ q_k \end{bmatrix} \in \mathbb{R}^{m+n}$, where $q_1, \ldots, q_n$ are the orthonormal vectors generated by the MGS process applied to A. We observe that

$$v_k^T v_k = (-e_k)^T(-e_k) + q_k^T q_k = 1 + 1 = 2$$

A standard Householder reflection is defined as

$$H = I - 2\frac{vv^T}{v^T v}$$

Substituting our expression for v_k, we obtain

$$P_k = I - 2\frac{v_k v_k^T}{2} = I - v_k v_k^T$$

For any $k \neq j$, the inner product $v_k^T v_j = e_k^T e_j + q_k^T q_j = 0 + 0 = 0$, because the standard basis vectors e_k and the MGS vectors q_k are respectively orthogonal.

Since these vectors are mutually orthogonal, the product of the Householder matrices simplifies to a summation: $P = P_1 P_2 \ldots P_n = I - \sum_{k=1}^{n} v_k v_k^T$.Next, we evaluate the block structure of P. The summation term is given by:

$$\sum_{k=1}^{n} v_k v_k^T = \sum_{k=1}^{n} \begin{bmatrix} -e_k \\ q_k \end{bmatrix} \begin{bmatrix} -e_k^T & q_k^T \end{bmatrix} = \sum_{k=1}^{n} \begin{bmatrix} e_k e_k^T & -e_k q_k^T \\ -q_k e_k^T & q_k q_k^T \end{bmatrix} = \begin{bmatrix} I_n & -Q^T \\ -Q & QQ^T \end{bmatrix}$$

where $Q = [q_1, \ldots, q_n]$. Subtracting this from the identity matrix $I_{m+n} = \begin{bmatrix} I_n & 0 \\ 0 & I_m \end{bmatrix}$, we find:

$$P = \begin{bmatrix} I_n - I_n & Q^T \\ Q & I_m - QQ^T \end{bmatrix} = \begin{bmatrix} 0_n & Q^T \\ Q & I_m - QQ^T \end{bmatrix}$$

This confirms the block form of P stated in the hint.Finally, we show that applying P^T (which equals P since it is symmetric) to the augmented matrix yields the upper triangular matrix R. Let $A = QR$ be the QR factorization of A via MGS. Then:

$$P^T \begin{bmatrix} 0_n \\ A \end{bmatrix} = \begin{bmatrix} 0_n & Q^T \\ Q & I - QQ^T \end{bmatrix} \begin{bmatrix} 0_n \\ QR \end{bmatrix} = \begin{bmatrix} Q^T(QR) \\ (I - QQ^T)QR \end{bmatrix} = \begin{bmatrix} (Q^TQ)R \\ QR - Q(Q^TQ)R \end{bmatrix}$$

Since the columns of Q are orthonormal, $Q^TQ = I_n$. Substituting this into the equation results in:

$$P^T \begin{bmatrix} 0_n \\ A \end{bmatrix} = \begin{bmatrix} I_nR \\ QR - QR \end{bmatrix} = \begin{bmatrix} R \\ 0 \end{bmatrix}$$

This proves that the Householder QR factorization of the augmented matrix $\begin{bmatrix} 0_n \\ A \end{bmatrix}$ is indeed equivalent to the MGS method applied to A, as it produces the same upper triangular matrix R and utilizes the MGS vectors q_k within the reflections.

Prob. 8

To verify the FLOP count for the Givens QR factorization of a matrix $A \in \mathbb{R}^{m \times n}$, we consider the cost of zeroing out each element below the main diagonal using successive Givens rotations. For each column j from 1 to n, we eliminate the elements a_{ij} for $i = j+1, \ldots, m$. A single Givens rotation $G(j, i, \theta)$ targeted at the element (i, j) requires calculating the rotation parameters $c = \cos\theta$ and $s = \sin\theta$, which involves a constant number of operations ($O(1)$) that is negligible in the overall asymptotic count. The primary computational cost lies in applying the rotation to the two affected rows, j and i, across the remaining columns $k = j+1, \ldots, n$.

For each such column k, the transformation is defined by:

$$\begin{bmatrix} a_{jk} \\ a_{ik} \end{bmatrix} \leftarrow \begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} a_{jk} \\ a_{ik} \end{bmatrix}$$

This update requires 4 multiplications and 2 additions per column, totaling 6 FLOPs per index k. Since there are $n - j$ columns to update after the pivot column, the cost for one rotation is approximately $6(n - j)$ FLOPs. Summing this over all required rotations to clear column j, we obtain the total cost:

$$\text{FLOPs} \approx \sum_{j=1}^{n} \sum_{i=j+1}^{m} 6(n - j) = \sum_{j=1}^{n} 6(m - j)(n - j)$$

To simplify the algebra, we perform a change of variables by letting $k = n - j$. As j ranges from 1 to n, the new index k ranges from $n - 1$ down to 0. We can rewrite the term $m - j$ as $(m - n) + (n - j) = (m - n) + k$.

Substituting these into the summation, we get:

$$6 \sum_{k=0}^{n-1} ((m - n) + k)k = 6 \left[(m - n) \sum_{k=0}^{n-1} k + \sum_{k=0}^{n-1} k^2 \right]$$

Using the standard power sum formulas $\sum_{k=1}^{N} k = \frac{N(N+1)}{2}$ and $\sum_{k=1}^{N} k^2 = \frac{N(N+1)(2N+1)}{6}$, we focus on the leading-order terms $N^2/2$ and $N^3/3$ where $N = n - 1 \approx n$. The expression becomes:

$$6 \left[(m - n)\frac{n^2}{2} + \frac{n^3}{3} \right] = 3(m - n)n^2 + 2n^3$$

Expanding the first term gives $3mn^2 - 3n^3$. Combining this with the $2n^3$ term results in the final leading-order count:

$$\text{FLOPs} \approx 3mn^2 - n^3$$

Prob. 9

```r
# CGS
cgs_qr <- function(A) {
  m <- nrow(A)
  n <- ncol(A)
  Q <- matrix(0, m, n)
  R <- matrix(0, n, n)
  for (j in 1:n) {
    v <- A[, j]
    if (j > 1) {
      for (i in 1:(j - 1)) {
        R[i, j] <- sum(Q[, i] * A[, j])
        v <- v - R[i, j] * Q[, i]
      }
    }
    R[j, j] <- sqrt(sum(v^2))
    Q[, j] <- v / R[j, j]
  }
  return(list(Q = Q, R = R))
}

# MGS
mgs_qr <- function(A) {
  m <- nrow(A)
  n <- ncol(A)
  V <- A
  Q <- matrix(0, m, n)
  R <- matrix(0, n, n)
  for (j in 1:n) {
    R[j, j] <- sqrt(sum(V[, j]^2))
    Q[, j] <- V[, j] / R[j, j]
    if (j < n) {
      for (k in (j + 1):n) {
        R[j, k] <- sum(Q[, j] * V[, k])
        V[, k] <- V[, k] - R[j, k] * Q[, j]
      }
    }
  }
  return(list(Q = Q, R = R))
}

# Householder
householder_qr <- function(A) {
  m <- nrow(A)
  n <- ncol(A)
  R <- A
  Q <- diag(m)
  for (k in 1:n) {
    x <- R[k:m, k]
    e1 <- c(1, rep(0, length(x) - 1))
    v <- sign(x[1]) * sqrt(sum(x^2)) * e1 + x
    v <- v / sqrt(sum(v^2))
```

```r
    R[k:m, k:n] <- R[k:m, k:n] - 2 * v %*% (t(v) %*% R[k:m, k:n])

    Q[, k:m] <- Q[, k:m] - 2 * (Q[, k:m] %*% v) %*% t(v)
  }
  return(list(Q = Q[, 1:n], R = R[1:n, 1:n]))
}

# Generate Vandermonde matrix
p <- seq(0, 1, length.out = 25)
A <- outer(p, 0:14, "^")

# Condition numbers
cond_A <- kappa(A, exact = TRUE)
cond_AtA <- kappa(t(A) %*% A, exact = TRUE)

# 3. Running decompositions and calculating errors
res_cgs <- cgs_qr(A)
res_mgs <- mgs_qr(A)
res_hh  <- householder_qr(A)

calc_errors <- function(A, res) {
  Q <- res$Q
  R <- res$R
  err_back <- sqrt(sum((A - Q %*% R)^2)) # ||A - QR||_F
  err_orth <- sqrt(sum((diag(ncol(Q)) - t(Q) %*% Q)^2)) # ||I - QtQ||_F
  return(c(BackErr = err_back, OrthErr = err_orth))
}

results <- rbind(
  CGS = calc_errors(A, res_cgs),
  MGS = calc_errors(A, res_mgs),
  Householder = calc_errors(A, res_hh)
)

print(list(Cond_A = cond_A, Cond_AtA = cond_AtA))
```

```
## $Cond_A
## [1] 36302856526
##
## $Cond_AtA
## [1] 6.354693e+17
```

```r
print(results)
```

```
##                  BackErr      OrthErr
## CGS         6.500331e-16 5.136001e+00
## MGS         6.152107e-16 8.440030e-07
## Householder 4.790879e-15 3.004886e-15
```